# GOOGLE DATA ANALYTICS: CAPSTONE PROJECT CYCLISTIC BIKE-SHARE ANALYSIS CASE STUDY

BY

MUHAMMAD MUSTHAFA

# INTRODUCTION

# INTRODUCTION

Cyclistic bike-share is a fictional company operating in Chicago, assuming myself as a junior data analyst working on the marketing analyst team at Cyclistic. The director of marketing believes the company's future success depends on maximizing the number of annual memberships. Therefore, the research team wants to understand how casual riders (customers) and annual members(subscribers) use Cyclistic bikes differently. From these insights, research team will design a new marketing strategy to convert casual riders into annual members.

For these, purpose I am responsible to collect and analyze historical data, and generate insights. For the purpose, I have collected the entire historical data which shows the total number of Subscriber and Customer riders with their starting and ending time and date for the year 2019 from the link given in the Google Data Analytics Course.

## OBJECTIVES

- To Understand riding habits of Customer and Subscriber Riders
- To Understand difference between the numbers of Subscribers and Customers
- To Understanding factors influencing the riders to choose the subscription plan

# METHODOLOGY

# METHODOLOGY

## Sources of Data

- The Data for this case study are downloaded from the link given in the Google Data Analytics

## Data Analysis Tool

Python Programming is used for the entire analysis for this study

**IDE:** Pycharm Community

**Libraries**: **Pandas** for data manipulation and analysis, **matplotlib** and\or seaborn for data visualization.

## Preprocessing Steps

### Data Transformation

- Merged Four quarter Dataset into one using concat function
- Pivoted data from rows to columns as necessary
- Aggregated data by categories such as Season, Months, Weekdays

### Data Cleaning

- Deleted unnecessary white spaces using strstrip function
- Transformed datatypes as necessary

### Data Quality Check

- Verified data types datatypes for each column
- Verified data ranges for numeric columns
- Ensured data consistency across rows and columns

## Python Code

```
import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.pyplot import title
from pandas import read_csv
df=pd.read_csv('C:/Users/MUHAMMAD
MUSTHAFA/Desktop/Data/Divvy_Trips_2019_Q1.csv')
df1=pd.read_csv('C:/Users/MUHAMMAD
MUSTHAFA/Desktop/Data/Divvy_Trips_2019_Q2.csv')
df2=pd.read_csv('C:/Users/MUHAMMAD
MUSTHAFA/Desktop/Data/Divvy_Trips_2019_Q3.csv')
df3=pd.read_csv('C:/Users/MUHAMMAD
MUSTHAFA/Desktop/Data/Divvy_Trips_2019_Q4.csv')
print(df2.columns)
print(df.columns)
print(df3.columns)
print(df1.columns)
df1.columns=['trip_id', 'start_time', 'end_time', 'bikeid', 'tripduration',
       'from_station_id', 'from_station_name', 'to_station_id',
       'to_station_name', 'usertype', 'gender', 'birthyear',]
print(df1.columns)
df_compined=pd.concat([df,df1,df2,df3])
print(df_compined)
print(df_compined.head(5))
print(df_compined.tail(5))
print(df_compined.iloc[:, :12].head())
df_compined['start_time']=pd.to_datetime(df_compined['start_time'],format='
%Y-%m-%d %H:%M:%S')
df_compined['starting_date']=df_compined['start_time'].dt.date
df_compined['starting_time']=df_compined['start_time'].dt.time
df_compined=df_compined.drop('start_time',axis=1)
print(df_compined)
df_compined['end_time']=pd.to_datetime(df_compined['end_time'],format='%Y-
%m-%d %H:%M:%S')
df_compined['ending_date']=df_compined['end_time'].dt.date
df_compined['ending_time']=df_compined['end_time'].dt.time
df_compined=df_compined.drop('end_time',axis=1)
print(df_compined)
df_compined=df_compined.drop(['trip_id','bikeid','from_station_id','gender'
,'birthyear','from_station_name','to_station_id','to_station_name'],axis=1)
df_compined['starting_date'] = pd.to_datetime(df_compined['starting_date'])
df_compined['ending_date'] = pd.to_datetime(df_compined['ending_date'])
df_compined['starting_day'] = df_compined['starting_date'].dt.day_name()
df_compined['starting_month'] =
df_compined['starting_date'].dt.month_name()
df_compined['ending_day'] = df_compined['ending_date'].dt.day_name()
df_compined['ending_month'] = df_compined['ending_date'].dt.month_name()
df_compined = df_compined.dropna(how='all')
df_compined['starting_time'] =
pd.to_datetime(df_compined['starting_time'],format='%H:%M:%S')
df_compined['ending_time'] =
pd.to_datetime(df_compined['ending_time'],format='%H:%M:%S')
df_compined['duration']=(df_compined['ending_time'] -
df_compined['starting_time']).dt.total_seconds() // 60
df_compined['duration']=df_compined['duration'].apply(lambda x:max(0,x))
df_compined['duration']=df_compined['duration'].clip(lower=0).astype(int)
df_compined=df_compined[df_compined['duration']!=0]
print(df_compined)
import pandas as pd
```

```
#group data by 'starting_day','usertype',then count
df_grouped=df_compined.groupby(['starting_day','usertype']).size().reset_in
dex(name='count')
#pivot data to have 'usertype' as columns
df_pivoted=df_grouped.pivot(index='starting_day',columns='usertype',values=
'count')
#plot data
day_order =
['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
df_pivoted = df_pivoted.loc[day_order]
df_pivoted.plot(kind='bar')
plt.show()
#count the number of customers and subscribers
subscriber_count=len(df_compined[df_compined['usertype'] == 'subscriber'])
customer_count = len(df_compined[df_compined['usertype'] == 'customer'])
##creating a list with counts
counts = [subscriber_count], [customer_count]
df_compined['usertype'] = df_compined['usertype'].str.strip()
print(df_compined['usertype'].unique())
#creating a list with the labels
label_values= ['subscriber'],['customer']
#plot the pie chart
import matplotlib.pyplot as plt
plt.figure(figsize=(10,8))
df_compined['usertype'].value_counts().plot(kind='pie',labels=label_values,
autopct='%1.1f%%')
plt.title('overall riders by usertype')
plt.show()
#creating a new column to catagorize rides by season
month_to_season= {
        'January':'winter',
        'February':'winter',
       'March':'spring',
       'April':'spring',
       'May':'spring',
       'June':'summer',
       'July':'summer',
       'August':'summer',
       'September':'fall',
       'October':'fall',
       'November':'fall',
       'December':'winter'
}
df_compined['season'] =
(df_compined['starting_month'].map(month_to_season))
#group the data by season and calculate the total rides
season_rides=df_compined.groupby('season') ['usertype'].count()
#plot the results
plt.figure(figsize=(10,6))
season_rides.plot(kind='bar')
plt.title('total rides by season')
plt.xlabel('season')
plt.ylabel('total rides')
plt.tight_layout()
plt.show()
#goup by season and usertype
season_riders=df_compined.groupby(['season', 'usertype']).size().unstack()
#plot the results
plt.figure(figsize=(10,6))
season_riders.plot(kind='bar')
plt.title('total rides by season and usertype')
```

```
plt.xlabel('season')
plt.ylabel('total rides')
plt.tight_layout()
plt.legend(title='usertype')
plt.show()
#pivating the data to create a matrix
pivot_df=df_compined.pivot_table(index='starting_month',columns='usertype',
aggfunc='size', fill_value=0)
print(df_compined.columns)
#creating a heatmap
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(10,8))
sns.heatmap(pivot_df, annot=True, cmap='Blues', fmt='d')
plt.title('Number of riders by month and usertype')
plt.xlabel('usertype')
plt.ylabel('starting_month')
plt.show()
import pandas as pdd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
df_compined['time'] = df_compined['starting_time'].dt.time
#creating a pivot table
df_pivate= df_compined.groupby(['time',
'usertype']).size().unstack('usertype')
#creating a chart
df_pivate.plot(kind='line',stacked=True, figsize=(10,6))
plt.gca().tick_params(axis='x', labelrotation=45, labelsize=8)
plt.tight_layout()
plt.title('Number of Riders by Start Time and User Type')
plt.xlabel('Start Time')
plt.ylabel('Number of Riders')
plt.legend()
plt.show()
#grouping the data
df_grouped=df_compined.groupby(['starting_day','usertype'])['duration'].mea
n().reset_index()
#pivoting data
df_Pivoted=df_grouped.pivot(index='starting_day',columns='usertype',values=
'duration')
#correct order of days
weekdays=['Monday','Tuesday','Wednesday','Thursday','Friday']
weekends=['Saturday','Sunday']
df_pivoted = df_pivoted.loc[weekdays + weekends]
#create chart
df_Pivoted.plot(kind='bar')
plt.title('Average Ride Duration by User Type and Day of week')
plt.xlabel('Day of week')
plt.ylabel('Average Ride Duration')
plt.legend(title='User Type')
for patch in plt.gca().patches:
       plt.text(patch.get_x(),patch.get_height(),
                str(round(patch.get_height(), 2)), ha='center' ,
va='bottom')
plt.show()
import matplotlib.pyplot as plt
import pandas as pd
from matplotlib.pyplot import title
from pandas import read_csv
df=pd.read_csv('C:/Users/MUHAMMAD
MUSTHAFA/Desktop/Data/Divvy_Trips_2019_Q1.csv')
```

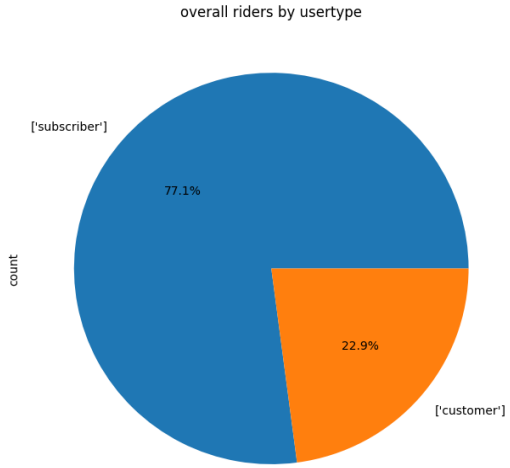
```
df1=pd.read_csv('C:/Users/MUHAMMAD
MUSTHAFA/Desktop/Data/Divvy_Trips_2019_Q2.csv')
df2=pd.read_csv('C:/Users/MUHAMMAD
MUSTHAFA/Desktop/Data/Divvy_Trips_2019_Q3.csv')
df3=pd.read_csv('C:/Users/MUHAMMAD
MUSTHAFA/Desktop/Data/Divvy_Trips_2019_Q4.csv')
print(df2.columns)
print(df.columns)
print(df3.columns)
print(df1.columns)
df1.columns=['trip_id', 'start_time', 'end_time', 'bikeid', 'tripduration',
       'from_station_id', 'from_station_name', 'to_station_id',
       'to_station_name', 'usertype', 'gender', 'birthyear',]
print(df1.columns)
df_compined=pd.concat([df,df1,df2,df3])
print(df_compined)
print(df_compined.head(5))
print(df_compined.tail(5))
print(df_compined.iloc[:, :12].head())
df_compined['start_time']=pd.to_datetime(df_compined['start_time'],format='
%Y-%m-%d %H:%M:%S')
df_compined['starting_date']=df_compined['start_time'].dt.date
df_compined['starting_time']=df_compined['start_time'].dt.time
df_compined=df_compined.drop('start_time',axis=1)
print(df_compined)
df_compined['end_time']=pd.to_datetime(df_compined['end_time'],format='%Y-
%m-%d %H:%M:%S')
df_compined['ending_date']=df_compined['end_time'].dt.date
df_compined['ending_time']=df_compined['end_time'].dt.time
df_compined=df_compined.drop('end_time',axis=1)
print(df_compined)
df_compined=df_compined.drop(['trip_id','bikeid','from_station_id','gender'
,'birthyear','from_station_name','to_station_id','to_station_name'],axis=1)
df_compined['starting_date'] = pd.to_datetime(df_compined['starting_date'])
df_compined['ending_date'] = pd.to_datetime(df_compined['ending_date'])
df_compined['starting_day'] = df_compined['starting_date'].dt.day_name()
df_compined['starting_month'] =
df_compined['starting_date'].dt.month_name()
df_compined['ending_day'] = df_compined['ending_date'].dt.day_name()
df_compined['ending_month'] = df_compined['ending_date'].dt.month_name()
df_compined = df_compined.dropna(how='all')
df_compined['starting_time'] =
pd.to_datetime(df_compined['starting_time'],format='%H:%M:%S')
df_compined['ending_time'] =
pd.to_datetime(df_compined['ending_time'],format='%H:%M:%S')
df_compined['duration']=(df_compined['ending_time'] -
df_compined['starting_time']).dt.total_seconds() // 60
df_compined['duration']=df_compined['duration'].apply(lambda x:max(0,x))
df_compined['duration']=df_compined['duration'].clip(lower=0).astype(int)
df_compined=df_compined[df_compined['duration']!=0]
print(df_compined)
import pandas as pd
#group data by 'starting_day','usertype',then count
df_grouped=df_compined.groupby(['starting_day','usertype']).size().reset_in
dex(name='count')
#pivot data to have 'usertype' as columns
df_pivoted=df_grouped.pivot(index='starting_day',columns='usertype',values=
'count')
#plot data
day_order =
['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday']
```

```
df_pivoted = df_pivoted.loc[day_order]
df_pivoted.plot(kind='bar')
plt.show()
#count the number of customers and subscribers
subscriber_count=len(df_compined[df_compined['usertype'] == 'subscriber'])
customer_count = len(df_compined[df_compined['usertype'] == 'customer'])
##creating a list with counts
counts = [subscriber_count], [customer_count]
df_compined['usertype'] = df_compined['usertype'].str.strip()
print(df_compined['usertype'].unique())
#creating a list with the labels
label_values= ['subscriber'],['customer']
#plot the pie chart
import matplotlib.pyplot as plt
plt.figure(figsize=(10,8))
df_compined['usertype'].value_counts().plot(kind='pie',labels=label_values,
autopct='%1.1f%%')
plt.title('overall riders by usertype')
plt.show()
#creating a new column to catagorize rides by season
month_to_season= {
        'January':'winter',
        'February':'winter',
       'March':'spring',
       'April':'spring',
       'May':'spring',
       'June':'summer',
       'July':'summer',
       'August':'summer',
       'September':'fall',
       'October':'fall',
       'November':'fall',
       'December':'winter'
}
df_compined['season'] =
(df_compined['starting_month'].map(month_to_season))
#group the data by season and calculate the total rides
season_rides=df_compined.groupby('season') ['usertype'].count()
#plot the results
plt.figure(figsize=(10,6))
season_rides.plot(kind='bar')
plt.title('total rides by season')
plt.xlabel('season')
plt.ylabel('total rides')
plt.tight_layout()
plt.show()
#goup by season and usertype
season_riders=df_compined.groupby(['season', 'usertype']).size().unstack()
#plot the results
plt.figure(figsize=(10,6))
season_riders.plot(kind='bar')
plt.title('total rides by season and usertype')
plt.xlabel('season')
plt.ylabel('total rides')
plt.tight_layout()
plt.legend(title='usertype')
plt.show()
#pivating the data to create a matrix
pivot_df=df_compined.pivot_table(index='starting_month',columns='usertype',
aggfunc='size', fill_value=0)
print(df_compined.columns)
```

```
#creating a heatmap
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(10,8))
sns.heatmap(pivot_df, annot=True, cmap='Blues', fmt='d')
plt.title('Number of riders by month and usertype')
plt.xlabel('usertype')
plt.ylabel('starting_month')
plt.show()
import pandas as pdd
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
df_compined['time'] = df_compined['starting_time'].dt.time
#creating a pivot table
df_pivate= df_compined.groupby(['time',
'usertype']).size().unstack('usertype')
#creating a chart
df_pivate.plot(kind='line',stacked=True, figsize=(10,6))
plt.gca().tick_params(axis='x', labelrotation=45, labelsize=8)
plt.tight_layout()
plt.title('Number of Riders by Start Time and User Type')
plt.xlabel('Start Time')
plt.ylabel('Number of Riders')
plt.legend()
plt.show()
#grouping the data
df_grouped=df_compined.groupby(['starting_day','usertype'])['duration'].mea
n().reset_index()
#pivoting data
df_Pivoted=df_grouped.pivot(index='starting_day',columns='usertype',values=
'duration')
#correct order of days
weekdays=['Monday','Tuesday','Wednesday','Thursday','Friday']
weekends=['Saturday','Sunday']
df_pivoted = df_pivoted.loc[weekdays + weekends]
#create chart
df_Pivoted.plot(kind='bar')
plt.title('Average Ride Duration by User Type and Day of week')
plt.xlabel('Day of week')
plt.ylabel('Average Ride Duration')
plt.legend(title='User Type')
for patch in plt.gca().patches:
       plt.text(patch.get_x(),patch.get_height(),
                str(round(patch.get_height(), 2)), ha='center' ,
va='bottom')
plt.show()
```

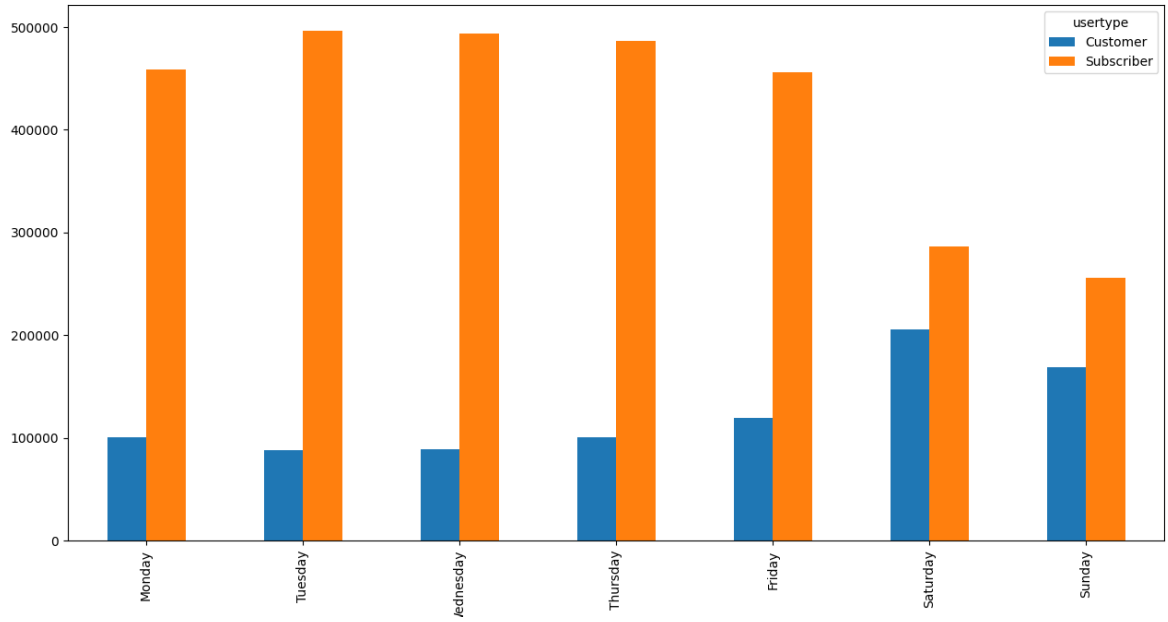
# RESULTS

# Overall Riders




From overall Riders 77.1% are subscribers and 22.9% are customers

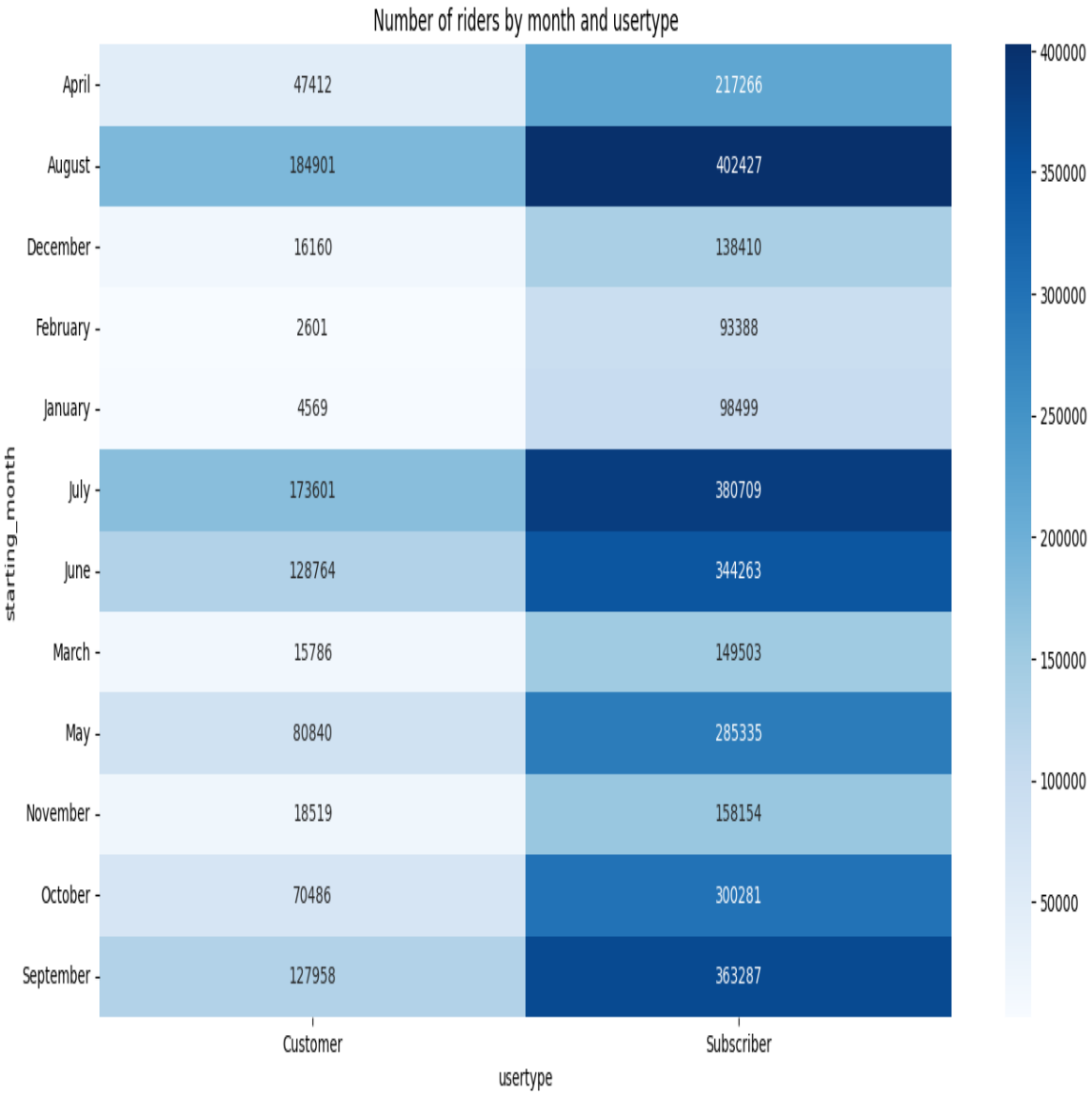
# DAILY RIDERS




## Key Findings

- Subscribers ride more during the weekdays than weekends
- Customers ride more during the weekends than weekdays

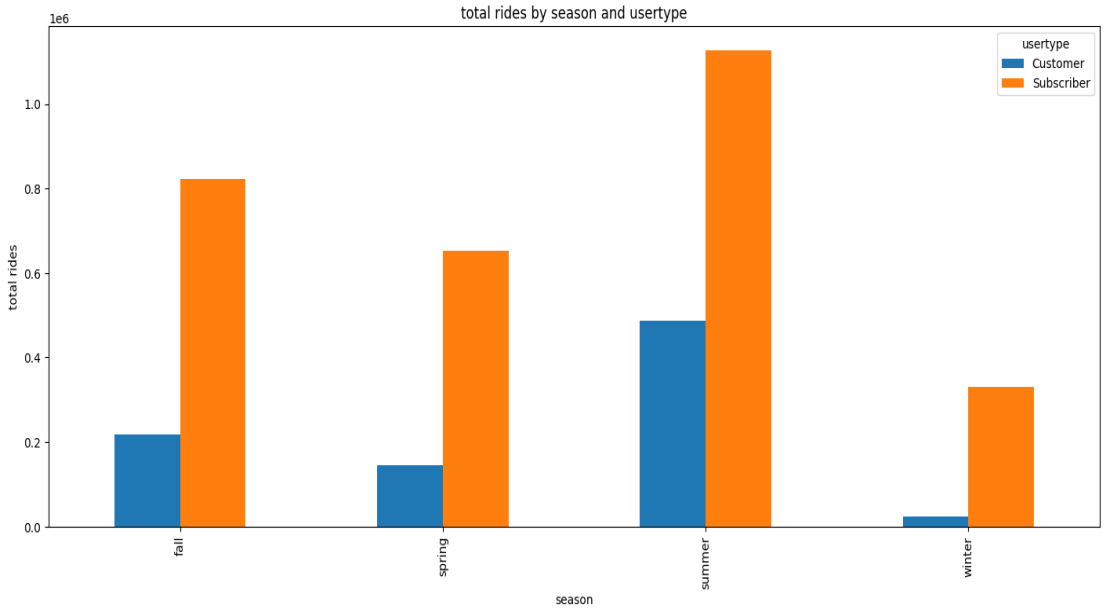
# MONTHLY RIDERS

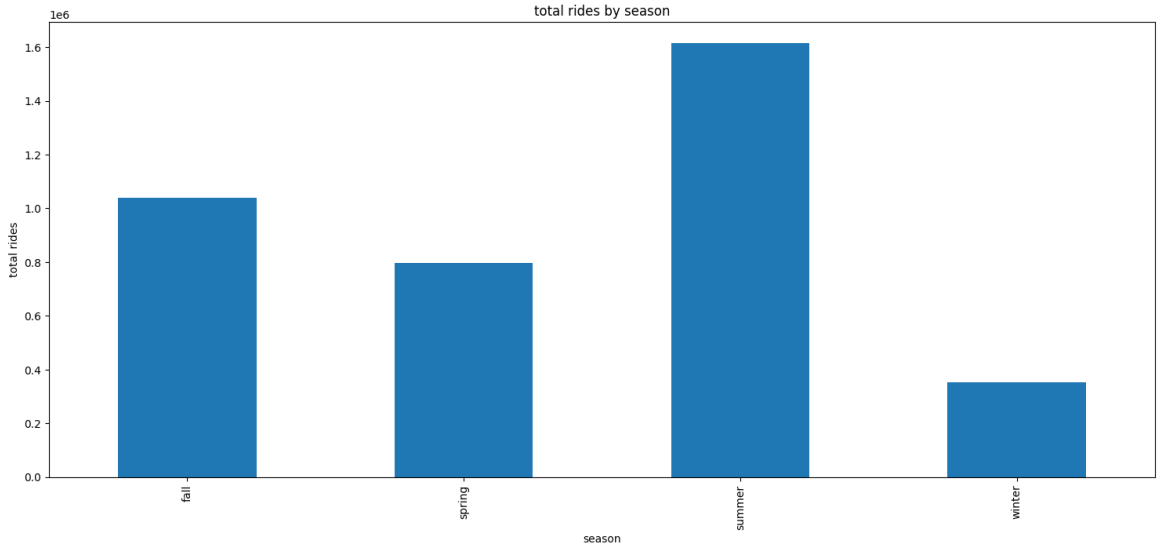



## Key Findings

- Month with the greatest number of riders is July
- Month with least number of riders is January
- Every month most of the riders are subscribers

# SEASONAL RIDERS

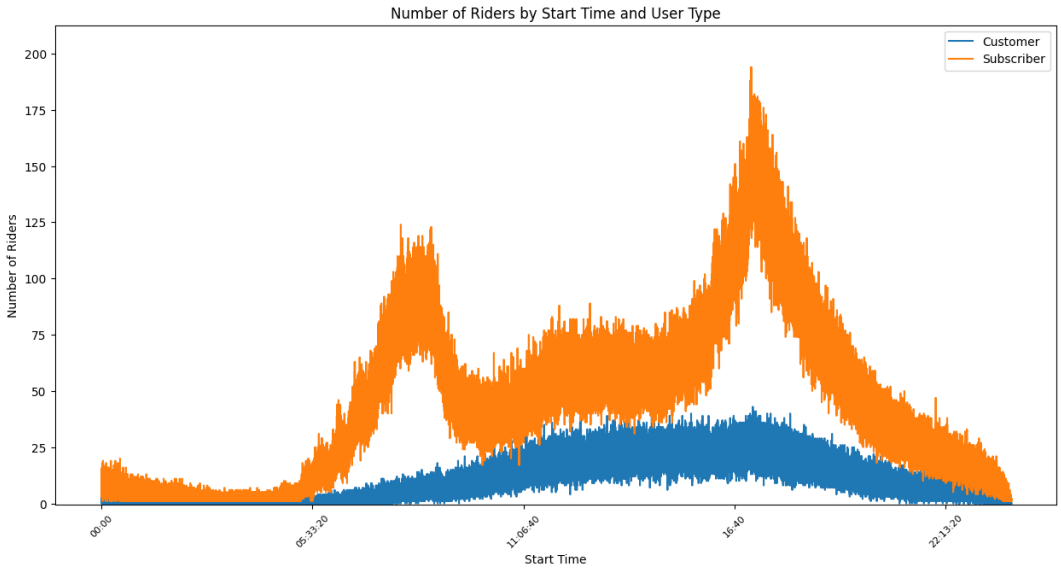



## Key Findings

- Summer has most riders
- Winter has least riders

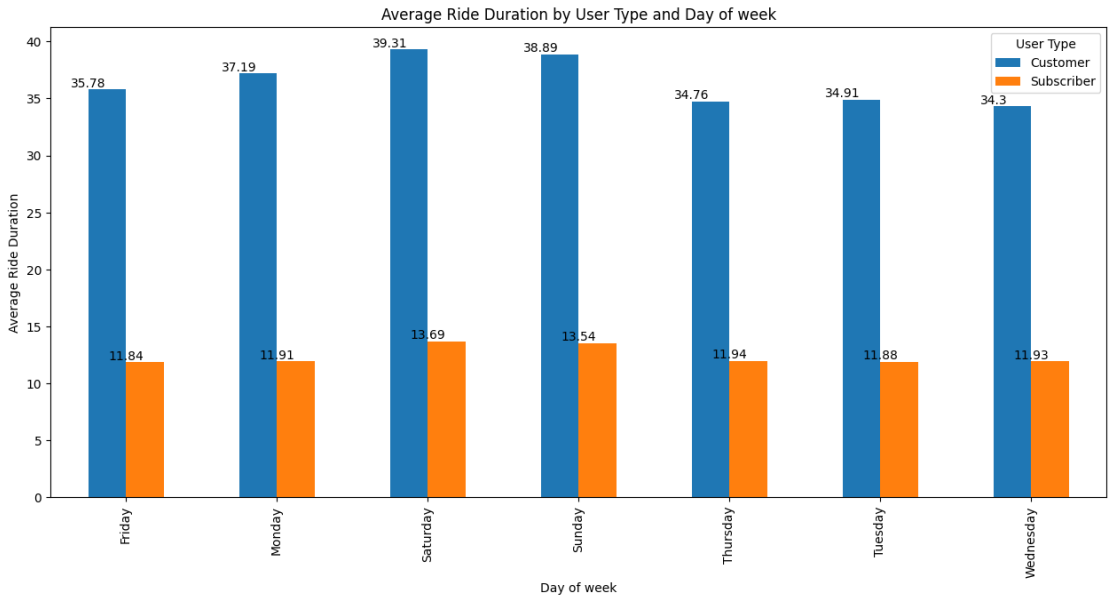
# STARTING TIME




## Key Findings

- Customers tend to progressively increase until the evening
- Subscribers rides correlate to the standard work time

# DURATION OF RIDES




## Key Findings

- Customers ride longer
- Subscribers' rides are lesser than 15 minutes

# INSIGHTS

- Most number of rides are occurring during the summer season
- Least number of rides are happening during the winter season
- Subscribers are riding more during the weekdays and starting time is correlating with standard working time, and duration is shorter
- Customers are appearing more on weekends and the duration is longer

# SUGGESSION AND CONCLUSION

# SUGESSIONS

- Most of the Subscribers are depending our bikes for the purpose of going to job Most of the Customers are riding our bikes for the purpose of spending the holydays, in order to convert those customers into subscribers we must implement attractive subscription plans for them.
- More rides occur on summer and least on winter, so we can slightly reduce charges during the winter and increase during summer.
- Make the annual subscription charge economical when comparing to the one time use charge.

# CONCLUSION

The objective of the study is to learn about how to convert customers into subscribers. The study was carried out by collecting historical data of year 2019.Still most of the riders are subscribers to our company. The study helped to know why other customers are staying unsubscribed and what to do to make them subscribe.